

FICHE DE RÉVISION – APPLICATIONS MOBILE (L3 Informatique)

Table des matières

1	Node.js / npm (BASE)	2
2	Serveur Express.js	2
3	Routes Express	3
4	HTML côté serveur	3
5	Fichiers statiques	3
6	Fetch (Front-end)	3
7	localStorage	4
8	DOM	4
9	Responsive Design	4
10	Menu responsive	5
11	Temps + expiration	5
12	Erreurs fréquentes	5
13	Résumé	5

1 Node.js / npm (BASE)

Qu'est-ce que Node.js ?

Node.js permet d'exécuter JavaScript en dehors du navigateur (côté serveur).

Commandes de base

```
node -v
npm -v
npm init -y
npm install express
```

Explication

Commande	Rôle
node -v	vérifier version Node.js
npm -v	vérifier version npm
npm init -y	créer projet Node
npm install express	installer Express

npm

npm = gestionnaire de paquets Il permet d'installer des bibliothèques.

Création :

- package.json → configuration projet
- *node_modules* dépendances installées

2 Serveur Express.js

```
const express = require("express")
const app = express()

app.use(express.json())

app.listen(5000, () => {
  console.log("Server running")
})
```

Explication

Ligne	Rôle
require("express")	importer framework
app = express()	créer serveur
express.json()	lire JSON
listen(5000)	lancer serveur

3 Routes Express

GET

```
app.get("/books", (req, res) => {  
  res.json(books)  
})
```

GET = lire données `res.json()` = envoyer JSON

POST

```
app.post("/books", (req, res) => {  
  books.push(req.body)  
  res.send("ok")  
})
```

POST = ajouter données `req.body` = données envoyées par client

4 HTML côté serveur

```
const path = require("path")  
  
res.sendFile(path.join(__dirname, "front/index.html"))
```

Explication

Élément	Rôle
<code>path</code>	gérer chemins fichiers
<code>dirname</code>	dossier actuel
<code>sendFile</code>	envoyer page HTML

5 Fichiers statiques

```
app.use('/css', express.static('node_modules/bootstrap/dist/css'))
```

Permet d'utiliser CSS/JS externes (Bootstrap)

6 Fetch (Front-end)

GET

```
fetch("/books")  
  .then(res => res.json())  
  .then(data => console.log(data))
```

POST

```
fetch("/books", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(book)
})
```

Partie	Rôle
method	type requête
headers	format données
JSON.stringify	convertir objet

7 localStorage

```
localStorage.setItem("books", JSON.stringify(books))
books = JSON.parse(localStorage.getItem("books"))
```

Explication

localStorage stocke les données dans le navigateur Il ne disparaît pas après refresh

Utilité :

- éviter fetch inutile
- garder données localement

8 DOM

```
const div = document.createElement("div")
div.className = "book-card"
container.appendChild(div)
```

Action	Rôle
createElement	créer élément HTML
className	ajouter style
appendChild	afficher dans page

9 Responsive Design

```
.book-card { width: 100%; }

@media (min-width: 679px) { width: 50%; }
@media (min-width: 923px) { width: 33.33%; }
```

Écran	Cartes
mobile	1
tablette	2
desktop	3

10 Menu responsive

```
btn.addEventListener("click", () => {  
    menu.classList.toggle("active")  
})
```

toggle() = ajouter/supprimer classe

11 Temps + expiration

```
localStorage.setItem("time", Date.now())  
  
const diff = Date.now() - lastTime
```

Conversion :

Temps	ms
1 sec	1000
1 min	60000
2 min	120000

12 Erreurs fréquentes

- oublier JSON.stringify
- oublier JSON.parse
- oublier res.json()
- mauvais guillemets
- classList = "... " (incorrect)

13 Résumé

- Backend = Express
- Frontend = fetch + DOM
- stockage = localStorage
- interface = CSS responsive